

Real-Time Sign Language Alphabet Recognition Using MobileNetV2 Transfer Learning

¹Akshay G

Department of MCA
Mangalore Institute of Technology & Engineering
Moodbidri, Mangalore, Karnataka, India
acharyaakshay367@gmail.com

²Dr. Anushree Raj

Department of MCA
Mangalore Institute of Technology & Engineering
Moodbidri, Mangalore, Karnataka, India
anushree@mite.ac.in

Abstract—People who are deaf or hard of hearing still face everyday barriers when speaking with others who do not know sign language. Assistive tools have improved over the years, yet a simple, low-cost bridge between signers and non-signers remains hard to achieve. Here we present a complete pipeline that classifies static American Sign Language (ASL) alphabet poses from a live camera feed. The classifier is built on MobileNetV2—a network sized for phones and edge devices—and extended to 29 labels: A–Z plus space, delete, and nothing. Training ran in two steps: first only the new top layers learned while ImageNet weights stayed fixed; then a few deeper blocks were opened for fine-tuning on hand shapes. From a balanced 8,700-image slice of an 87,000-image public set, the model reached 90.68% on training data and 84.29% on validation after 11 epochs. On ten held-out test photos, nine were labeled correctly; high-confidence calls often scored above 99%. An OpenCV webcam demo prints recognized letters on screen without a GPU or special gloves.

Keywords—*Sign Language Recognition, MobileNetV2, Transfer Learning, Convolutional Neural Networks, Deep Learning, Real-Time Inference, Assistive Technology, Computer Vision*

I. INTRODUCTION

World Health Organization figures suggest about 430 million people live with disabling hearing loss today, and that number may pass 700 million before 2050. Sign languages are rich, complete languages for deaf communities, but most hearing people never learn them. The mismatch shows up in clinics, schools, shops, and other places where an interpreter is usually not available.

Recent progress in deep learning and computer vision has made camera-based sign language recognition (SLR) more realistic for student projects and field trials. Using only a webcam avoids datagloves and body-worn sensors, which cuts cost and setup time. Convolutional neural networks (CNNs) map hand shape and finger layout in still frames well enough to support alphabet-level recognition without extra hardware.

Starting a large CNN from random weights for fine-grained classes needs huge labeled sets and long GPU runs. A common shortcut is transfer learning: load weights trained on a general vision task such as ImageNet, then retrain the top layers (and sometimes selected blocks) on sign images. In our experience this converges sooner and holds up better on modest datasets than training the full stack from scratch.

Our system centers on MobileNetV2 because it was built for fast inference on CPUs and low-memory devices. Phase one froze the pretrained backbone and trained only the new classification head; phase two unfroze selected deep layers so the network could adapt to ASL hand poses. The trained weights were wrapped in a simple OpenCV loop that reads webcam frames and shows predicted letters as text overlay.

II. LITERATURE REVIEW

Published SLR work dates back many years and covers gloves, depth cameras, and plain RGB video. The papers summarized in this section either shaped our design choices or offer accuracy numbers we use later when comparing results.

A. Dataglove for Sign Language Recognition via Wearable Inertial Sensors

Ji et al. [1] fitted a glove with 16 inertial units on fingers and palm. They passed IMU readings—angular rate, acceleration, and orientation—into an Attention-BiLSTM and reported 98.85% accuracy on 20 gestures. Accuracy is strong, but users must buy and wear custom hardware, which limits everyday adoption.

B. A Comparative Review on Applications of Different Sensors for SLR

Amin, Rizvi, and Hossain [2] reviewed over one hundred SLR papers and grouped them by whether they used cameras, wearables, or both. Combining modalities often helped robustness, while vision-only setups were cheaper but more sensitive to lighting and background clutter. That trade-off led us to add rotation, zoom, and flip augmentation during training.

C. Sign Language Recognition System

Kalsh and Garewal [3] showed early webcam-based recognition using color segmentation, edge features, and k-nearest neighbors on 2-D hand gestures. Results were acceptable in a controlled studio but fell when lighting changed—an issue later CNN-based systems handle more gracefully.

D. Sign Language to Voice Translator

Sai Praneetha et al. [4] aimed to turn ASL poses into speech. MediaPipe supplied hand landmarks; a CNN handled alphabet letters while simple geometry rules covered digits. Letters and numbers followed separate paths before pyttsx3 spoke the combined string.

E. Real-Time Lightweight SLR on Hybrid Deep CNN-BiLSTM with Attention

Kazbekova et al. [5] used a CNN per frame plus a bidirectional LSTM across time, with attention to emphasize useful frames. On Sign Language MNIST they reported strong F1 scores and faster training than plain LSTM baselines. Their setup assumes GPU inference, whereas our goal was acceptable speed on a laptop CPU only.

F. Night Guardian: Empowering Safety with Hand Sign-Based Communication and AI

Mohana Priya and Ignatius [6] focused on a small set of emergency hand signs for safety apps. They compared SVM, KNN, Random Forest, and a stacked ensemble on MediaPipe landmarks; the stack worked best in dim light. The study shows gesture input is useful outside translation, for example in alert systems.

G. Advancements and Challenges in Hand Gesture Recognition: A Comprehensive Review

Murad and Alasadi [7] surveyed static and dynamic gesture methods and sorted them by what spatial or temporal cues they use. ANNs and HMMs often topped dynamic-sequence benchmarks, with many papers claiming above 90% accuracy. They also note that different datasets and metrics make side-by-side comparison difficult—a point we keep in mind when citing others.

H. Real-Time Hand Gesture Recognition for Healthcare and Security Systems

Mohamed et al. [8] tested 3-D CNNs and vision transformers on continuous gesture streams. Their HGR-ViT model reached 99.85% on standard benchmarks and they argued that motion patterns can act as a contact-free biometric harder to replay than a still photo.

III. METHODOLOGY

A. Dataset Description

We used a public ASL RGB dataset with 87,000 images in 29 folders—3,000 images per class. Classes are A–Z plus space (word break), delete (backspace intent), and nothing (no active sign) [8,9,10]. Photos were taken indoors with similar but not identical lighting; hand pose and angle vary so the model must rely on hand shape rather than fixed background.

For faster iteration we drew a stratified subset of 8,700 images (300 per class), keeping class ratios while reducing training time by about tenfold [11]. All numbers in this paper refer to that subset unless stated otherwise.

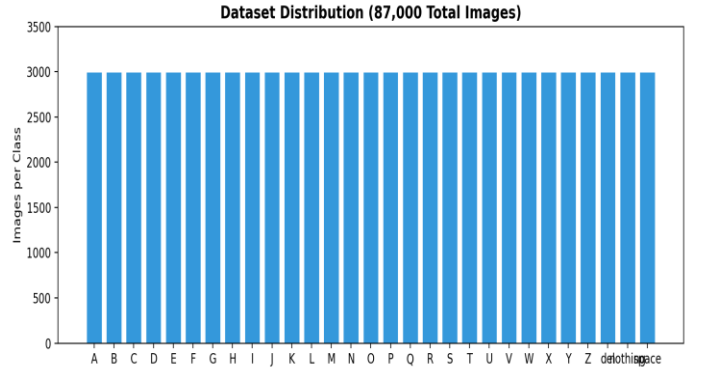


Fig. 1. Class-wise distribution of images across the 29 sign language categories

B. Model Architecture

We chose MobileNetV2 from TensorFlow's model zoo as the backbone. Inverted residuals and linear bottlenecks keep multiply-adds low while retaining useful features. We started from ImageNet weights rather than random initialization so early layers already encode edges, textures, and parts useful for hands.

A custom head sits on the backbone. Inputs are 128×128 RGB images [12]. During training we apply on-the-fly augmentation: random horizontal flip, up to 10% zoom, and rotation within $\pm 10^\circ$. Features pass through MobileNetV2 (2,257,984 parameters), then global average pooling to a 1,280-D vector. Dropout (0.3), a 256-unit ReLU dense layer, another dropout, and a 29-way softmax produce class probabilities [13]. Of 2,593,373 total parameters, 335,389 were trainable in phase one.

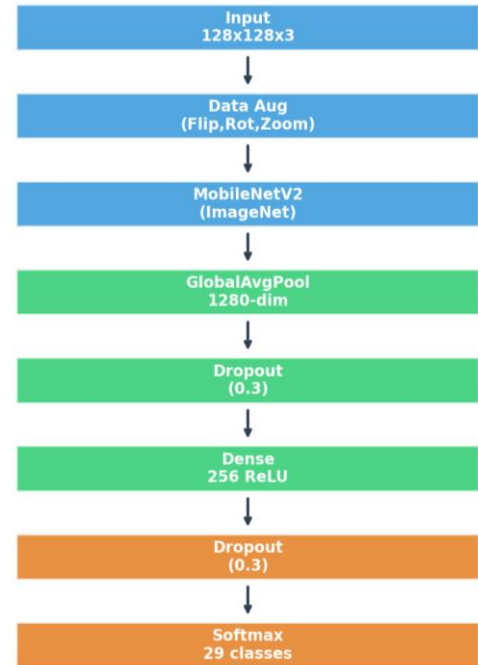


Fig. 2. Architecture of the proposed model showing the MobileNetV2 backbone and appended classification head

C. Training Strategy

Fine-tuning a pretrained CNN too aggressively can erase useful low-level filters—a problem often called catastrophic forgetting [14]. We therefore split training into two phases so updates ramp up gradually.

Phase one kept all 154 MobileNetV2 layers frozen so gradients updated only the 335,389 parameters in the new head [15]. We ran eight epochs with Adam at learning rate 1×10^{-3} and batch size 64, letting the head learn sign patterns before touching backbone weights. Phase two unfroze layers after index 100 and trained three more epochs at 1×10^{-4} so pretrained filters shifted slowly rather than being replaced outright. Fifteen percent of training images were held out for validation throughout; optimization used categorical cross-entropy.

TABLE I. TRAINING HYPERPARAMETERS

Parameter	Value
Base Model	MobileNetV2 (ImageNet)
Input Size	$128 \times 128 \times 3$
Optimizer	Adam
Phase 1 LR	0.001
Phase 2 LR	0.0001
Batch Size	64
Total Epochs	11 (8 + 3)
Loss Function	Categorical Cross-Entropy

D. Real-Time Inference Pipeline

The trained model was plugged into an OpenCV webcam loop. Frames are mirrored horizontally so users see a natural mirror view. A 250×250 region of interest in the center marks where hands should appear; it is resized to 128×128 , converted from BGR to RGB, and normalized [16]. To limit CPU load, inference runs every fourth frame. A character is written only when confidence exceeds 60% and the same class appears twice in a row, which filters out one-frame glitches.

IV. RESULTS AND DISCUSSION

A. Training Performance

All eleven epochs completed in about 15.5 minutes on an Intel CPU without a GPU. In phase one, training accuracy rose from 27.91% (epoch 1) to 74.46% (epoch 8) while validation reached 89.43%. The gap between train and validation accuracy during this stage is expected: the frozen backbone already supplies strong features, so validation reflects head generalization before the smaller head starts to overfit.

Opening deeper layers in phase two briefly dropped training accuracy to 69.56% at epoch 9 while the optimizer adapted to more trainable weights at the lower learning rate. By epoch 11 training accuracy peaked at 90.68%, validation at 84.29%, and validation loss at 0.5050. The six-point train-validation gap matches mild overfitting on the 8,700-image subset rather than the full 87,000-image corpus.

Epoch	Train Loss	Train Acc (%)	Val Loss	Val Acc (%)
1	2.5817	27.91	1.1319	71.11
2	1.5136	53.71	0.6881	82.84
3	1.2135	61.69	0.5410	84.83
4	1.0654	66.03	0.4619	85.29
5	0.9737	68.72	0.4323	86.74
6	0.8783	71.74	0.3726	88.28
7	0.8503	72.28	0.3891	87.66
8	0.7909	74.46	0.3240	89.43
9	0.9544	69.56	0.5948	81.30
10	0.4157	86.38	0.5660	82.61
11	0.2844	90.68	0.5050	84.29

Sample	True	Predicted	Conf (%)	Result
J425.jpg	J	J	100.00	Correct
X904.jpg	X	X	99.26	Correct
D1749.jpg	D	D	100.00	Correct
T1101.jpg	T	X	50.09	Incorrect
U491.jpg	U	U	99.71	Correct
U1594.jpg	U	U	99.03	Correct
F1979.jpg	F	F	100.00	Correct
del252.jpg	del	del	100.00	Correct
T2664.jpg	T	T	62.91	Correct
G784.jpg	G	G	99.79	Correct

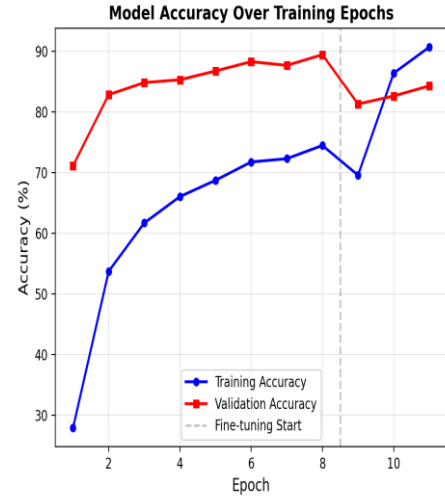


Fig. 3. Training and validation accuracy curves plotted across all 11 epochs

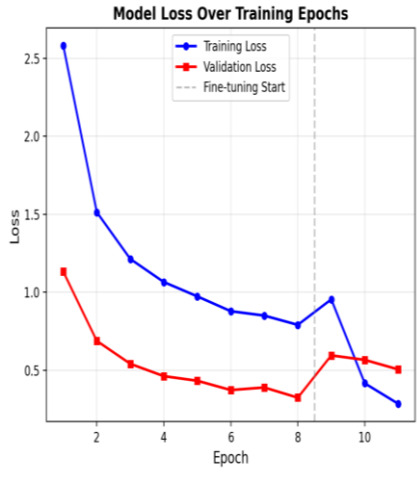


Fig. 4. Training and validation loss curves plotted across all 11 epochs

B. Random Sample Testing

We also tested ten random images from files not used in training or validation. Nine were classified correctly (90% accuracy). Seven of those nine had softmax scores above 99%, so the model is decisive when the pose is clear.

The remaining image was true class T but predicted as X at 50.09% confidence. Both signs use a closed fist with the thumb near the fingers; only thumb position differs. The low score is useful: the network signaled uncertainty instead of a wrong high-confidence label, which a future voting or temporal rule could exploit

. RANDOM SAMPLE TEST RESULTS

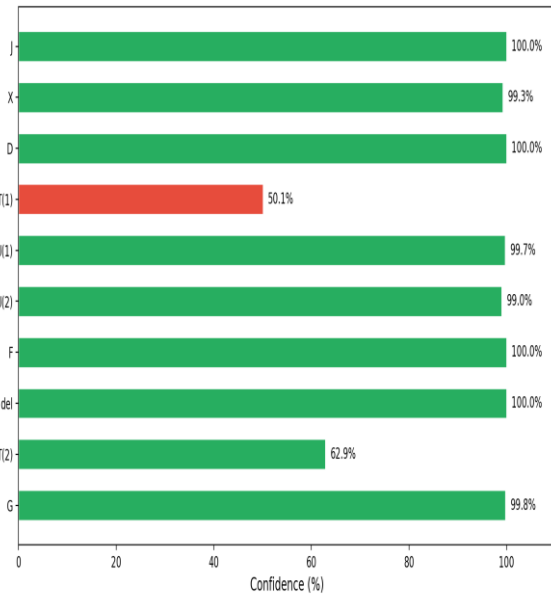


Fig. 5. Prediction confidence scores for the ten random test samples

C. Comparative Analysis

Against related studies, our design favors deployability over maximum accuracy. Kalsh and Garewal (2013) reached similar rates with kNN and edges but needed controlled studio light. Ji

et al. (2023) hit 98.85% with a 16-sensor glove most users would not wear daily. Kazbekova et al. (2025) gain from CNN-BiLSTM over frame sequences but need a GPU for real-time use. Our contribution is a commodity webcam and CPU-only inference without extra peripherals.

COMPARISON WITH EXISTING APPROACHES

Study	Method	Accuracy	Hardware
Kalsh & Garewal (2013)	kNN + Edge Detection	High (controlled)	Webcam
Ji et al. (2023)	Attention-BiLSTM	98.85%	Data Glove (16 IMUs)
Kazbekova et al. (2025)	CNN-BiLSTM+Attn	SOTA	Camera+GPU
Mohamed et al. (2024)	HGR-ViT	99.85%	Camera+GPU
Proposed System	MobileNetV2	90.68%/84.29 %	Webcam (CPU)

V. CONCLUSION

We showed that MobileNetV2 with two-phase transfer learning can power a real-time ASL alphabet recognizer on ordinary hardware. On the 8,700-image subset the network separated 29 gesture classes with 90.68% training and 84.29% validation accuracy; blind tests on ten external images yielded 90% correct labels, mostly at very high confidence—important if the tool is meant to support communication.

Splitting training into freeze-then-fine-tune phases mattered: early epochs protected ImageNet features while the head learned, and the later low-rate pass aligned deeper layers with hand geometry. The saved model is about 28.6 MB, runs live on a standard CPU, and needs only a webcam.

Next steps could add LSTM or transformer models for multi-frame words, train on more lighting and skin-tone diversity, and compress weights for phones via pruning or quantization. User studies with deaf and hard-of-hearing participants would also show whether lab accuracy translates to daily conversation settings.

REFERENCES

- [1] Ji, A., Wang, Y., Miao, X., Fan, T., Ru, B., Liu, L., Nie, R., & Qiu, S. (2023). Dataglove for Sign Language Recognition via Wearable Inertial Sensors.
- [2] Amin, M. S., Rizvi, S. T. H., & Hossain, M. M. (2022). A Comparative Review on Applications of Different Sensors for Sign Language Recognition.
- [3] Kalsh, A., & Garewal, N. S. (2013). Sign Language Recognition System.
- [4] Sai Praneetha, B., Dyanesh, R., Praniksha, M., Sanjai, R. J., & Singh, A. K. (2025). Sign Language to Voice Translator.
- [5] Kazbekova, G., Ismagulova, Z., Ibrayeva, G., Sundetova, A., Abdrazakh, Y., & Baimurzaev, B. (2025). Real-Time Lightweight SLR on Hybrid Deep CNN-BiLSTM with Attention.
- [6] Mohana Priya S., & Ignatius, A. G. (2024). Night Guardian: Empowering Safety with Hand Sign-Based Communication and AI.
- [7] Raj, T. N., & Deepak, G. (2025). Silent Alert: Advancing Security Through Smart Sign Recognition and AI.
- [8] Murad, B. K., & Alasadi, A. H. H. (2023). Advancements and Challenges in Hand Gesture Recognition: A Comprehensive Review.
- [9] Mohamed, A. S., Hassan, N. F., & Jamil, A. S. (2024). Real-Time Hand Gesture Recognition for Healthcare and Security Systems.

- [10] Hasan, M. M., & Misra, P. K. (2012). Brightness Factor Matching for Gesture Recognition System Using Scaled Normalization.
- [11] Devika, A. Raj, P. M. O and K. Adamsab, "AI-Powered Vision System Using OpenCV and MediaPipe for Real-Time Silent Emergency Hand Gesture Recognition in Privacy-Constrained Spaces for Women's Safety," pp. 601-606, doi: 10.1109/INSPIRE67328.2025.11300627. .
- [12] Howard, A. G. et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861
- [13] MediaPipe Hands. (2020). MediaPipe Hands: On-device Real-time Hand Tracking. Google Research.
- [14] Molchanov, P., Yang, X., Gupta, S., et al. (2016). Online Detection and Classification of Dynamic Hand Gestures with Recurrent 3D Convolutional Neural Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
- [15] M O, P., P, H., & Raj, A. (2024). Smart Facial Recognition with Age Estimation, Gender Classification and Emotion Detection. QTanalytics Publication (Books), 11–23. <https://doi.org/10.48001/978-81-966500-7-0-2> .
- [16] Oyedotun, O. K., & Khashman, A.. (2017). Deep Learning in Vision-Based Static Hand Gesture Recognition. Neural Computing and Applications, 28(12), 3941–3951.